Scenario: "PustokBazar" Online Bookstore Database

You are the database developer for an online bookstore. The system tracks **customers**, **books**, **orders**, and the **items within each order**. Use the schema and sample data below to answer all questions. Write your queries in standard SQL (MySQL syntax).

Table 1: `Customers`

CustomerID (PK)	Name	City	JoinDate
1	Rahim Uddin	Dhaka	2024-01-10
2	Karim Ahmed	Chattogram	2024-03-22

Table 2: `Books`

BookID (PK)	Title	Category	Price	Stock
101	Database Systems	Technology	850	20
102	Bangla Kobita Shomogro	Literature	400	5
103	Clean Code	Technology	950	0
104	Economics of Bangladesh	Business	600	15
105	Networking Basics	Technology	700	8

Table 3: `Orders`

OrderID (PK)	CustomerID (FK)	OrderDate	Status
1001	1	2026-06-01	Delivered
1002	2	2026-06-05	Delivered

Table 4: `OrderItems`

OrderItemID (PK)	OrderID (FK)	BookID (FK)	Quantity
1	1001	101	1

2	1001	105	2
3	1002	102	1

Q1. Write a query to display the `Title`, `Category`, and `Price` of all books priced above **600**, sorted by `Price` in descending order. *(10 marks)*

Q2. Write a query to display the number of customers who joined from each `City`, showing only cities with **more than 1** customer. *(10 marks)*

Q3. Write a query using a subquery to find the `Title` of the book(s) with the **highest price** in the `Books` table (without using `ORDER BY ... LIMIT`). *(10 marks)*

Q4. Write a query using a **correlated subquery** to list all customers (`Name`) who have placed **at least one order** with a `Status` of `'Delivered'`. *(10 marks)*

Q5. Write a query using `INNER JOIN` to display the `OrderID`, customer `Name`, and `OrderDate` for all orders placed by customers from `'Dhaka'`. *(10 marks)*

Q6. Write a query using a `LEFT JOIN` to list every book's `Title` along with the total `Quantity` ordered across all orders — including books that have **never been ordered** (their total should show as 0 or NULL). *(10 marks)*

Q7. Write a query joining all four tables (`Customers`, `Orders`, `OrderItems`, `Books`) to display: customer `Name`, `OrderID`, book `Title`, `Quantity`, and the line total (`Quantity × Price`), for orders with `Status = 'Delivered'`. *(10 marks)*

Q8. Write a **Stored Procedure** named `GetOrdersByCustomer` that accepts a `CustomerID` as an input parameter and returns all orders (`OrderID`, `OrderDate`, `Status`) placed by that customer. *(10 marks)*

Q9. Write a **Stored Procedure** named `UpdateStock` that accepts a `BookID` and a `Quantity` as input parameters, and reduces that book's `Stock` by the given quantity — but only if enough stock is available (otherwise it should not update and should indicate insufficient stock). *(10 marks)*

Q10. Create a **View** named `DeliveredOrderSummary` that shows `CustomerName`, `OrderID`, `OrderDate`, and the total order value (sum of `Quantity × Price` for all items in that order), limited to orders with `Status = 'Delivered'`. Then write a query that uses this view to find the customer with the **highest total single-order value**. *(10 marks)*